

Database Systems

LAB#08



23K-2001

BCS-5J

Q1:

ACCEPT empno PROMPT 'Enter employee number: '

DECLARE

v_sal employees.salary%TYPE;

v_bonus NUMBER;

BEGIN

SELECT salary INTO v_sal FROM employees WHERE employee_id = &empno;

IF v_sal IS NULL THEN

v_bonus := 0;

ELSIF v_sal < 1000 THEN

v_bonus := v_sal * 0.10;

ELSIF v_sal BETWEEN 1000 AND 1500 THEN

v_bonus := v_sal * 0.15;

ELSE

v_bonus := v_sal * 0.20;

END IF;

DBMS_OUTPUT.PUT_LINE('Bonus = ' || v_bonus);

END;

The screenshot displays the Oracle SQL Developer interface. The main window shows a PL/SQL script in the 'Query Builder' tab. The script is as follows:

```
ELSIF v_sal BETWEEN 1000 AND 1500 THEN
    v_bonus := v_sal * 0.15;
ELSE
    v_bonus := v_sal * 0.20;
END IF;

DBMS_OUTPUT.PUT_LINE('Bonus = ' || v_bonus);
END;
```

The 'Script Output' window shows the execution results:

```
Task completed in 2.799 seconds
v_bonus := v_sal * 0.20;
END IF;

DBMS_OUTPUT.PUT_LINE('Bonus = ' || v_bonus);
END;

PL/SQL procedure successfully completed.
```

The 'Dbms Output' window shows the final output:

```
Bonus = 1640
```

Q2:

ACCEPT empno PROMPT 'Enter employee number: '

DECLARE

v_comm employees.commission_pct%TYPE;

BEGIN

SELECT commission_pct INTO v_comm

FROM employees

WHERE employee_id = &empno;

IF v_comm IS NULL THEN

UPDATE employees

SET salary = salary + 0

WHERE employee_id = &empno;

DBMS_OUTPUT.PUT_LINE('Commission was NULL. Salary adjusted.');

ELSE

DBMS_OUTPUT.PUT_LINE('Commission is not NULL. No update needed.');

END IF;

END;

The screenshot displays the Oracle SQL Developer interface. The main window shows a PL/SQL script in the 'Query Builder' tab. The script is as follows:

```
UPDATE employees
SET salary = salary + 0
WHERE employee_id = &empno;

DBMS_OUTPUT.PUT_LINE('Commission was NULL. Salary adjusted.');
```

The 'Script Output' window shows the execution results:

```
Task completed in 6.356 seconds

DBMS_OUTPUT.PUT_LINE('Commission was NULL. Salary adjusted.');
```

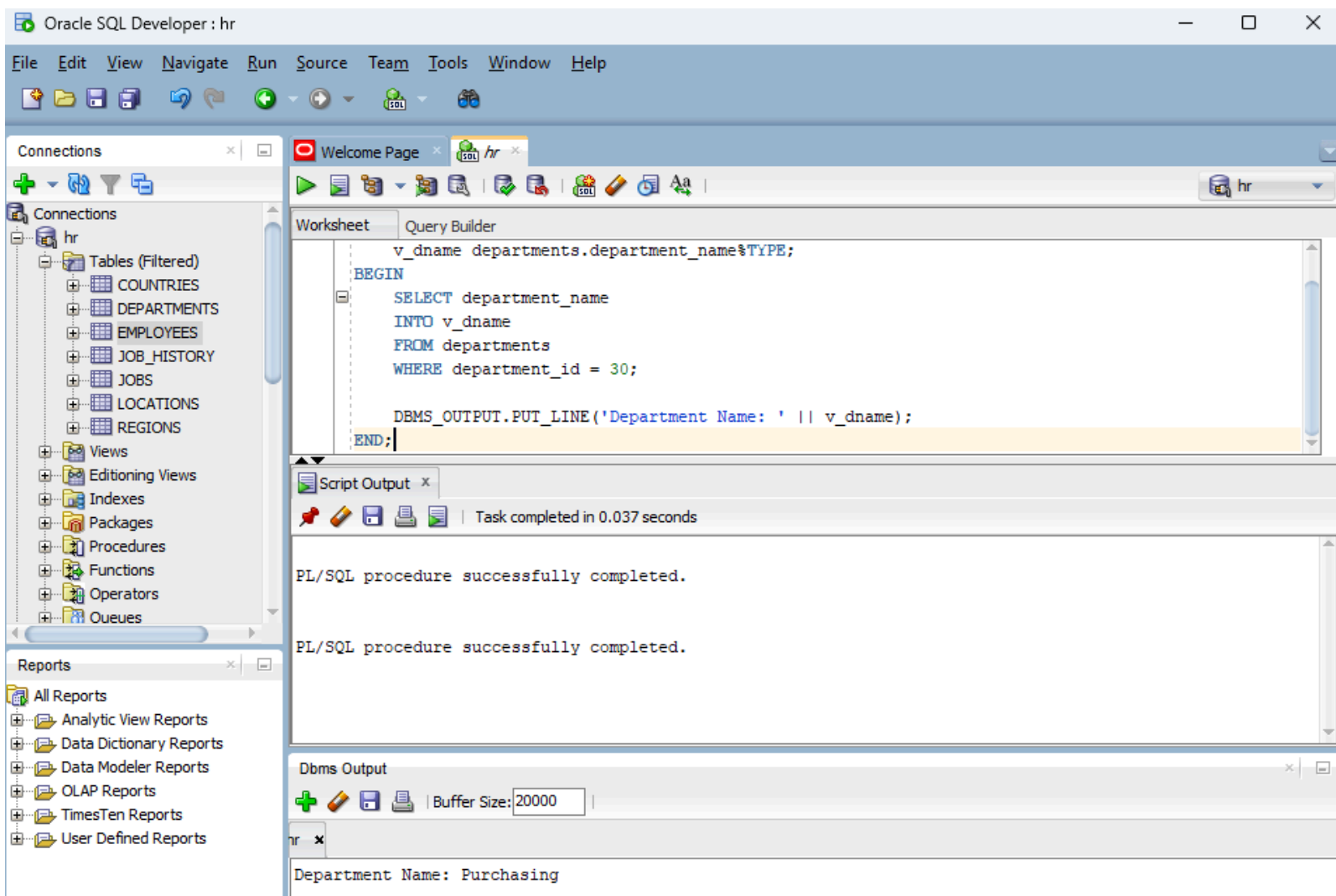
The 'Dbms Output' window shows the final output:

```
Commission was NULL. Salary adjusted.
```

Q3:

```
DECLARE
    v_dname departments.department_name%TYPE;
BEGIN
    SELECT department_name
    INTO v_dname
    FROM departments
    WHERE department_id = 30;

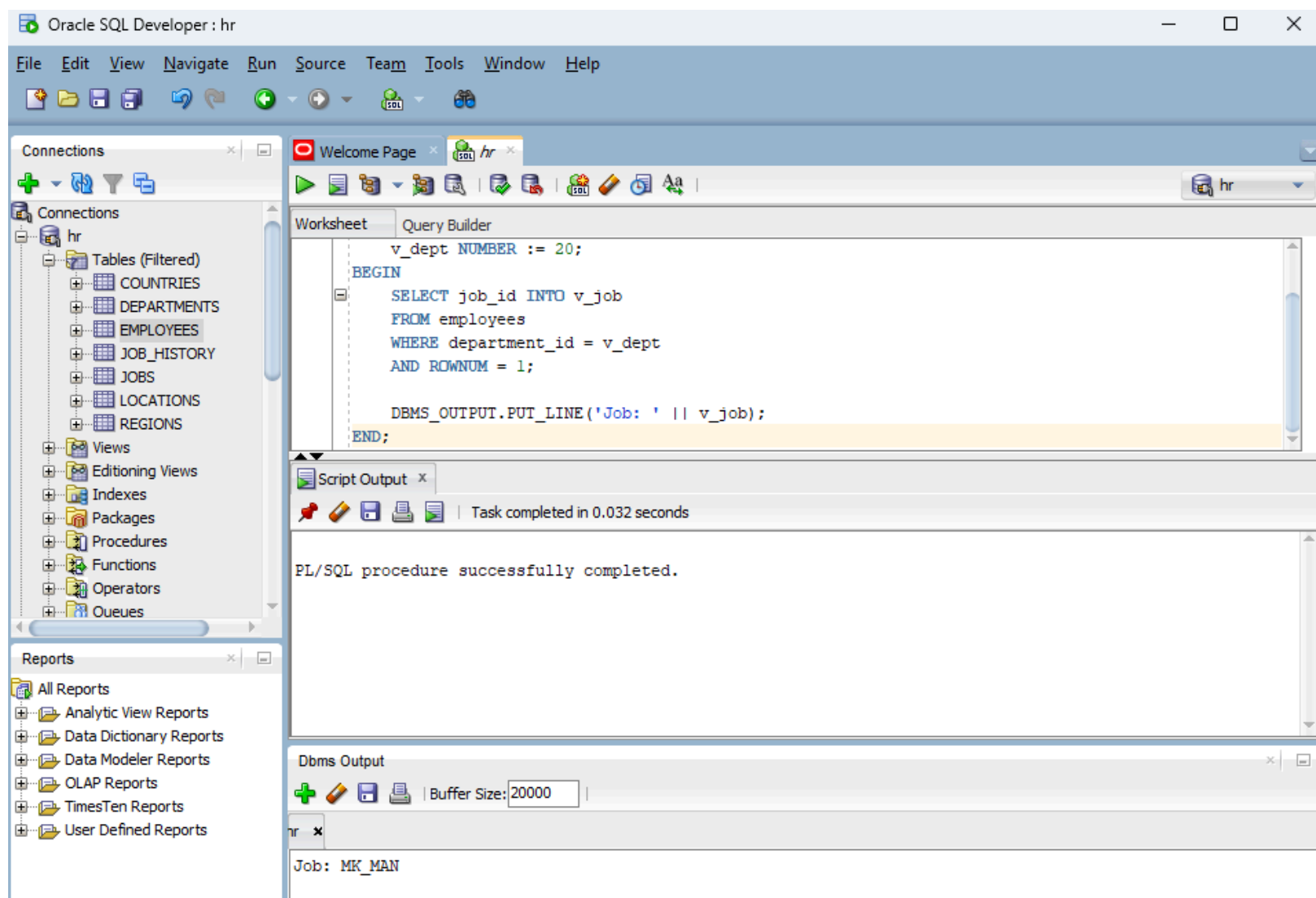
    DBMS_OUTPUT.PUT_LINE('Department Name: ' || v_dname);
END;
```



Q4:

```
DECLARE
    v_job employees.job_id%TYPE;
    v_dept NUMBER := 20;
BEGIN
    SELECT job_id INTO v_job
    FROM employees
    WHERE department_id = v_dept
    AND ROWNUM = 1;

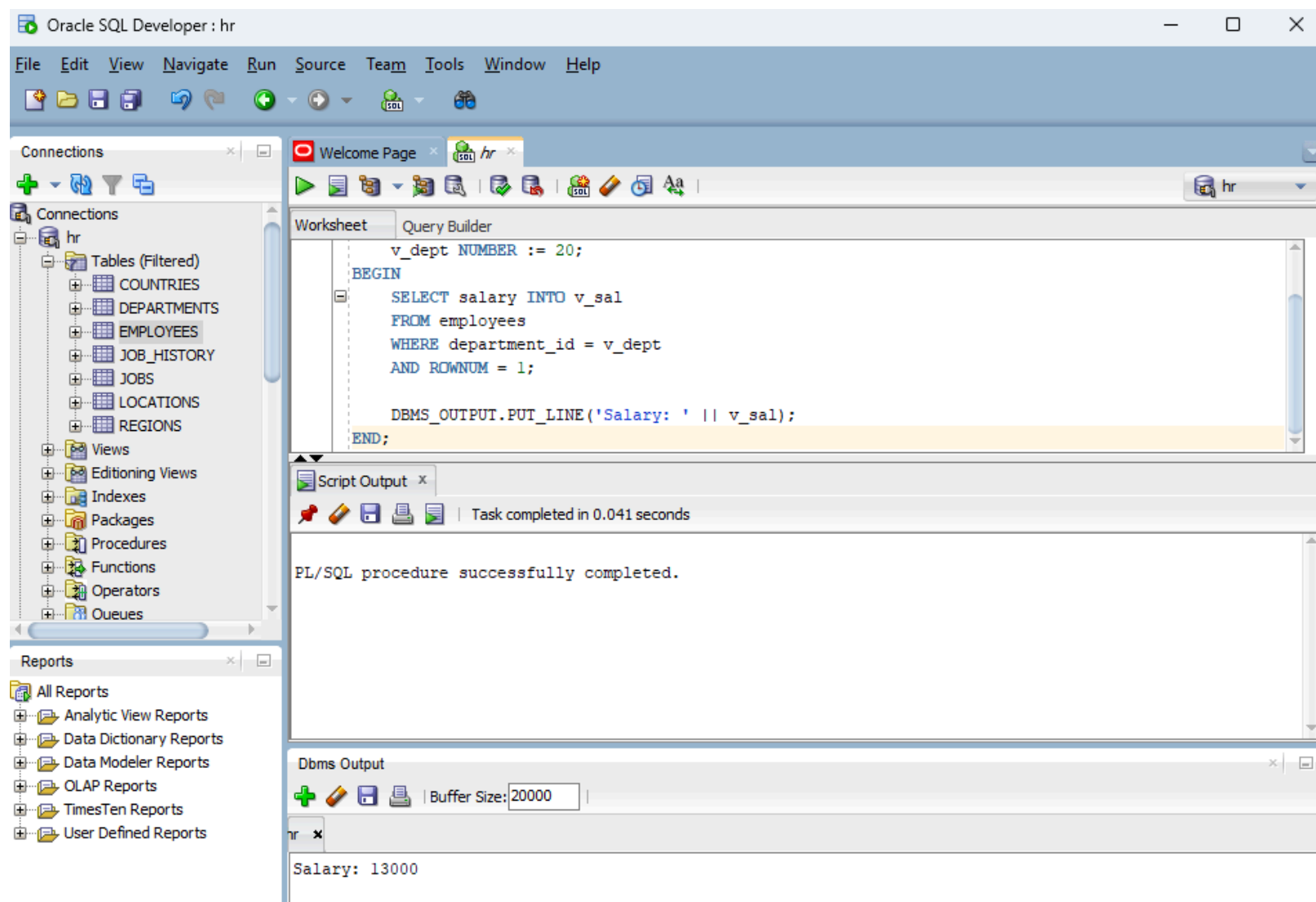
    DBMS_OUTPUT.PUT_LINE('Job: ' || v_job);
END;
```



Q5:

```
DECLARE
    v_sal employees.salary%TYPE;
    v_dept NUMBER := 20;
BEGIN
    SELECT salary INTO v_sal
    FROM employees
    WHERE department_id = v_dept
    AND ROWNUM = 1;

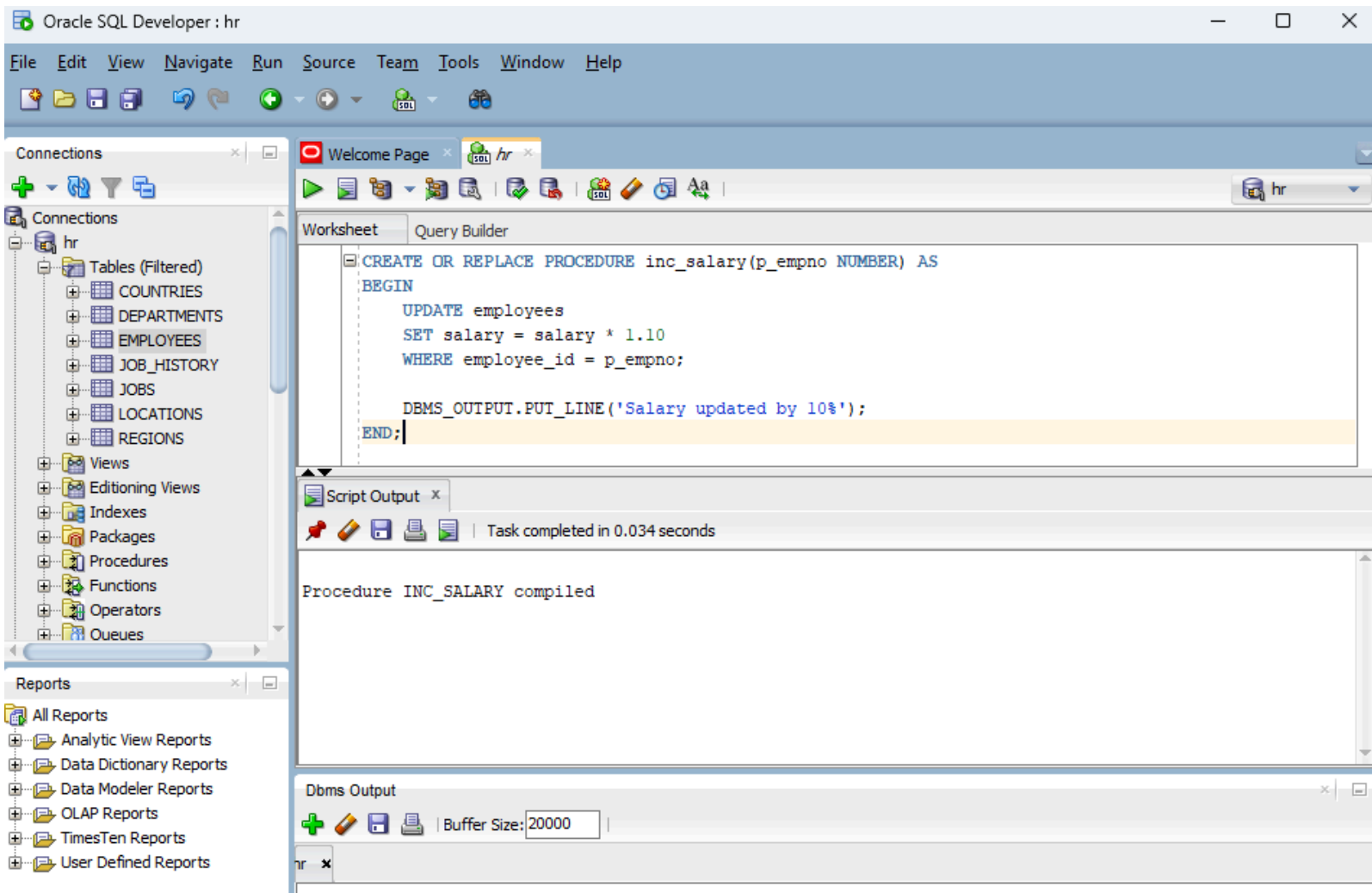
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_sal);
END;
```



Q6:

```
CREATE OR REPLACE PROCEDURE inc_salary(p_empno NUMBER) AS
BEGIN
    UPDATE employees
    SET salary = salary * 1.10
    WHERE employee_id = p_empno;

    DBMS_OUTPUT.PUT_LINE('Salary updated by 10%');
END;
```

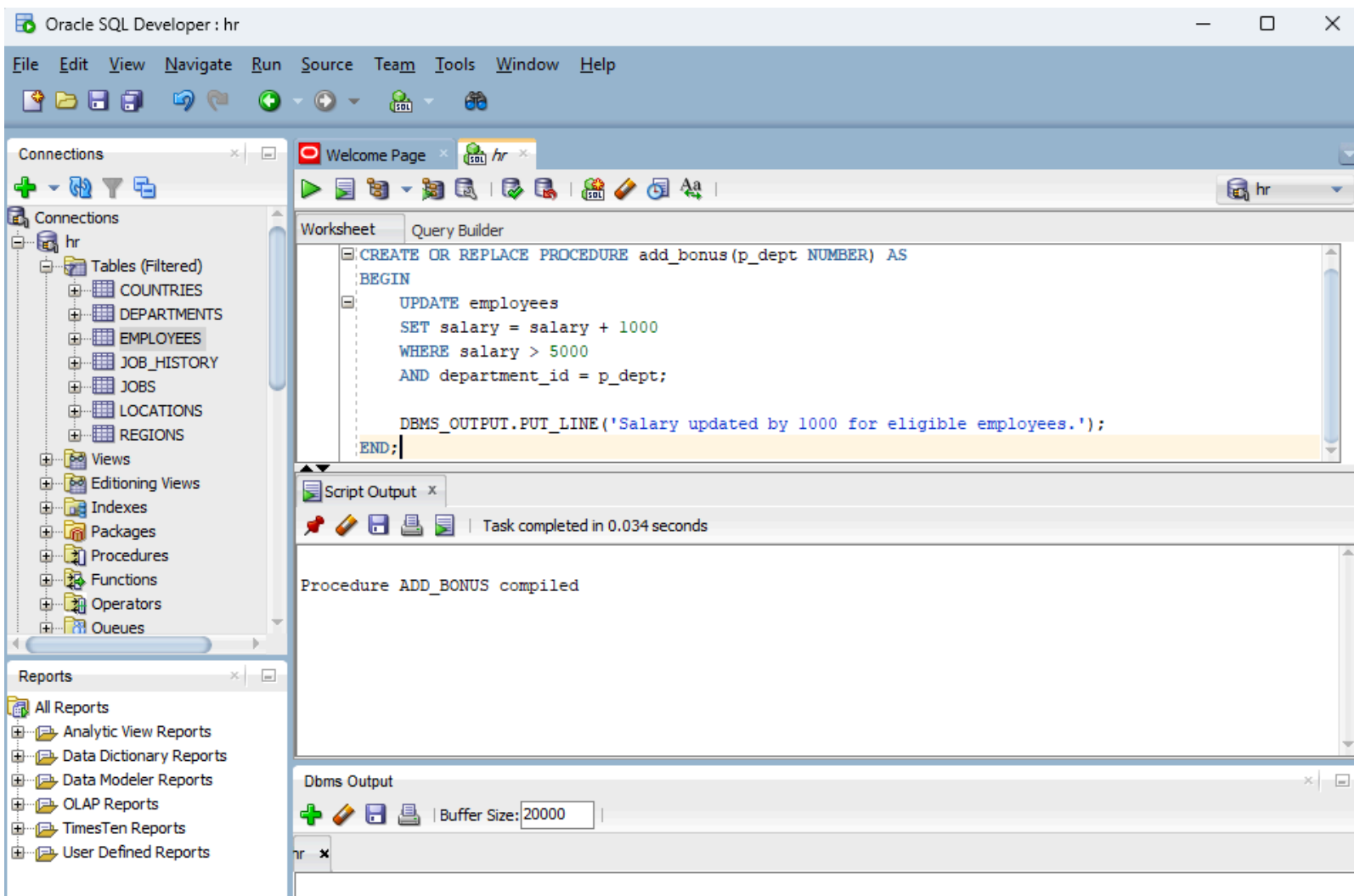


Q7:

```
CREATE OR REPLACE PROCEDURE add_bonus(p_dept NUMBER) AS
BEGIN
    UPDATE employees
    SET salary = salary + 1000
    WHERE salary > 5000
    AND department_id = p_dept;

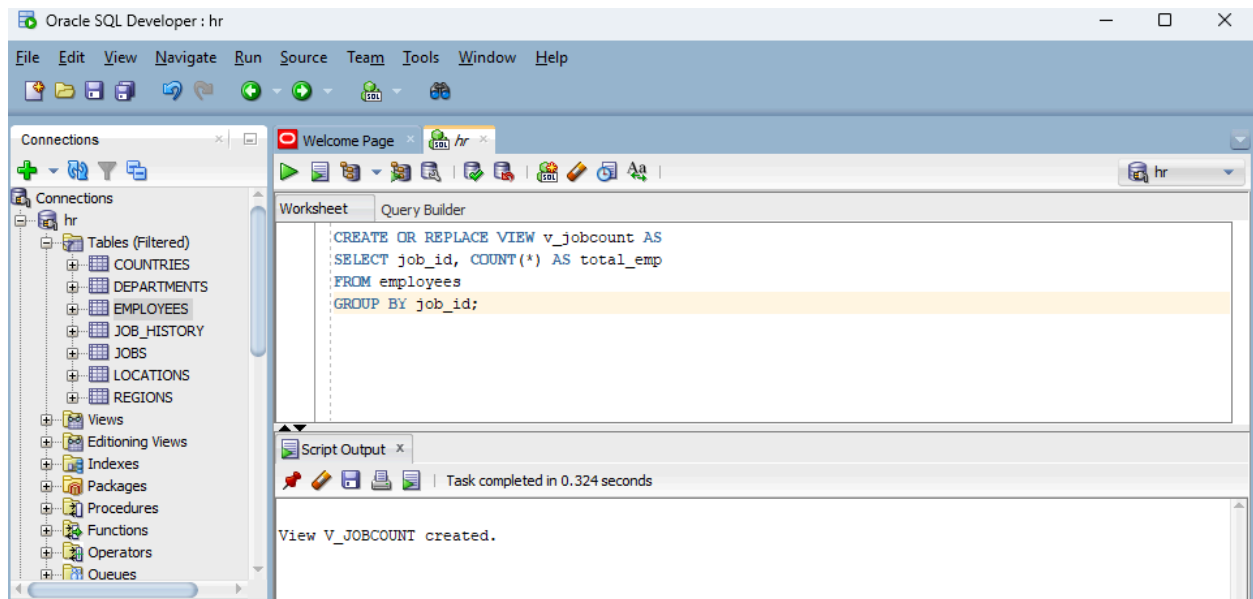
    DBMS_OUTPUT.PUT_LINE('Salary updated by 1000 for eligible employees.');
```

END;

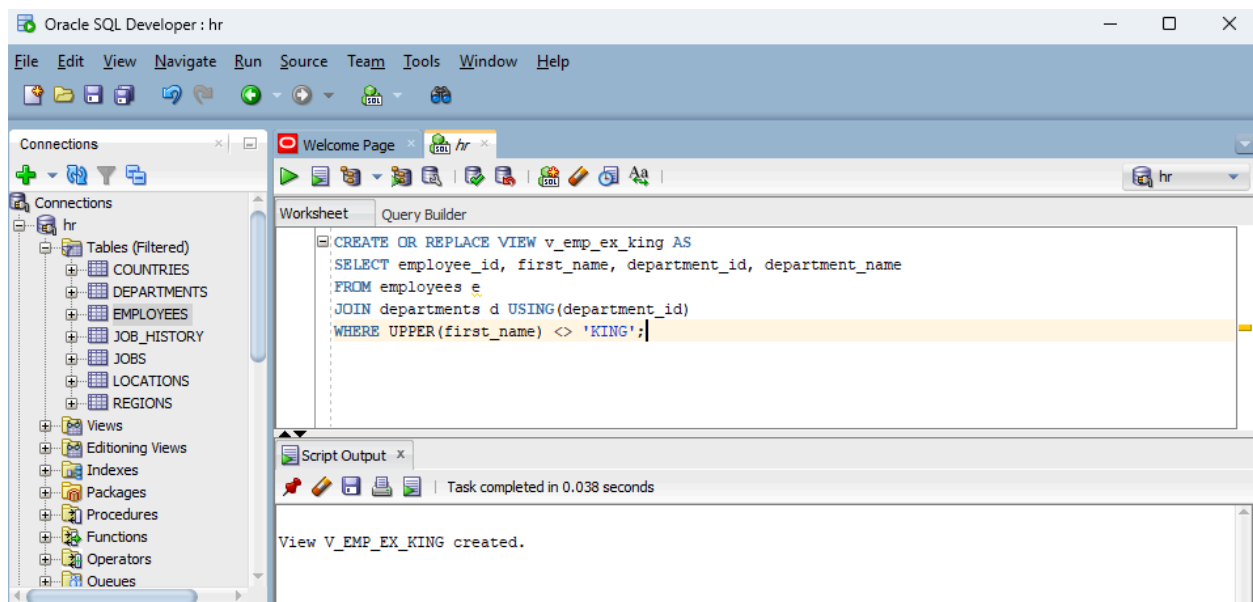


Q8:

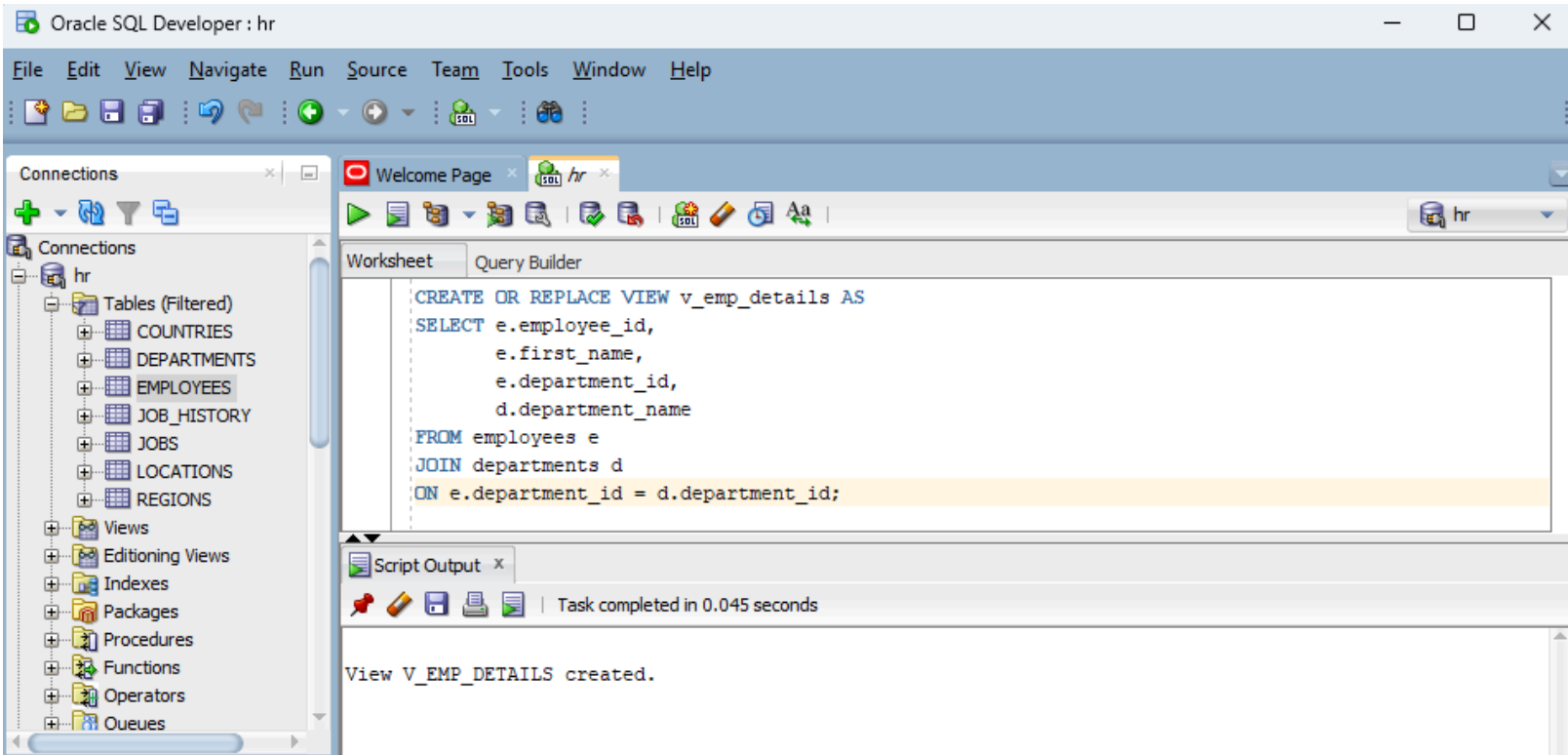
```
CREATE OR REPLACE VIEW v_jobcount AS
SELECT job_id, COUNT(*) AS total_emp
FROM employees
GROUP BY job_id;
```



```
CREATE OR REPLACE VIEW v_emp_ex_king AS
SELECT employee_id, first_name, department_id, department_name
FROM employees e
JOIN departments d USING(department_id)
WHERE UPPER(first_name) <> 'KING';
```



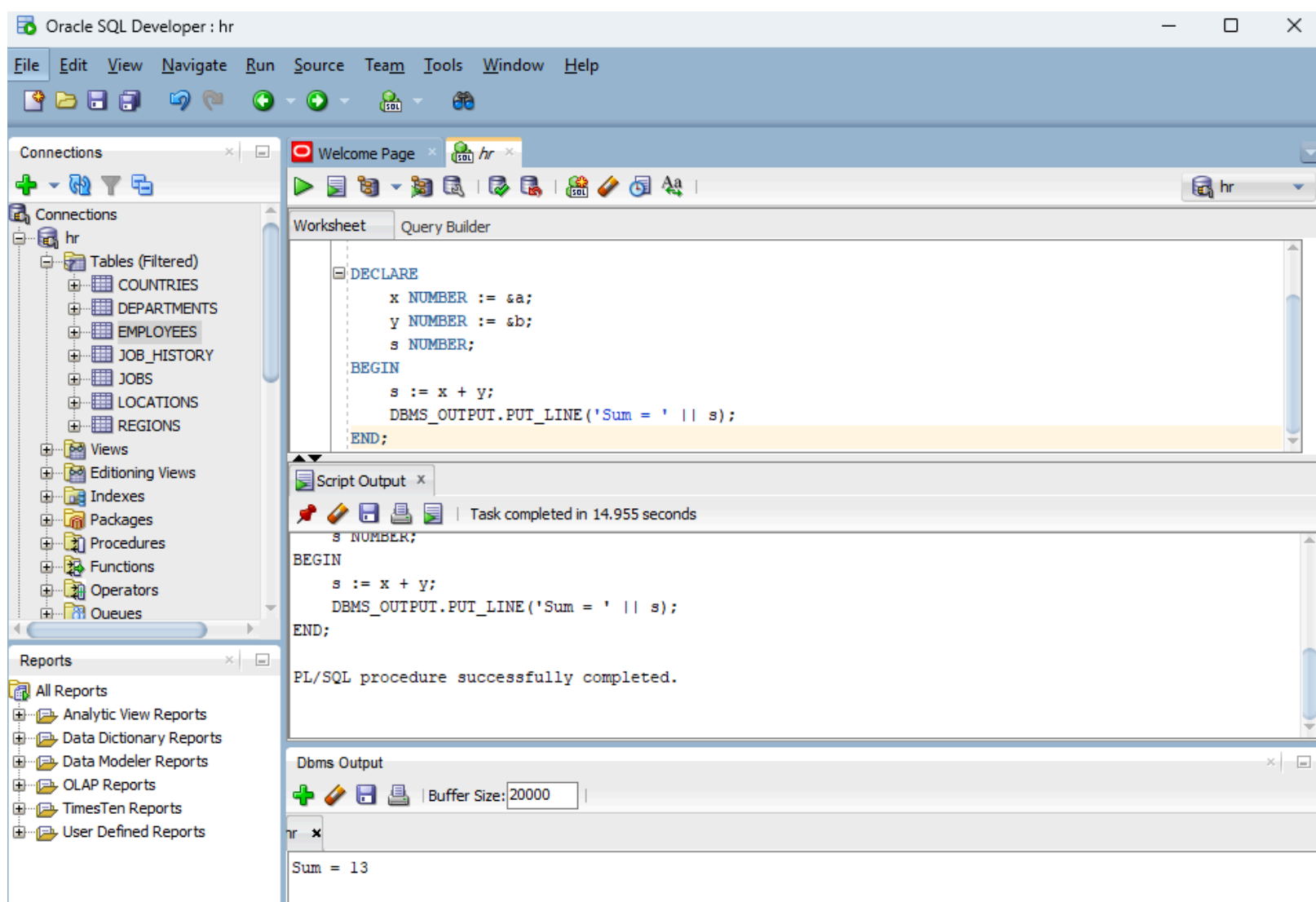
```
CREATE OR REPLACE VIEW v_emp_details AS
SELECT e.employee_id,
       e.first_name,
       e.department_id,
       d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```



Q9:

ACCEPT a PROMPT 'Enter first number: '
ACCEPT b PROMPT 'Enter second number: '

```
DECLARE
  x NUMBER := &a;
  y NUMBER := &b;
  s NUMBER;
BEGIN
  s := x + y;
  DBMS_OUTPUT.PUT_LINE('Sum = ' || s);
END;
```



Q10:

ACCEPT l PROMPT 'Enter lower boundary: '
ACCEPT u PROMPT 'Enter upper boundary: '

DECLARE

low NUMBER := &l;

high NUMBER := &u;

total NUMBER := 0;

BEGIN

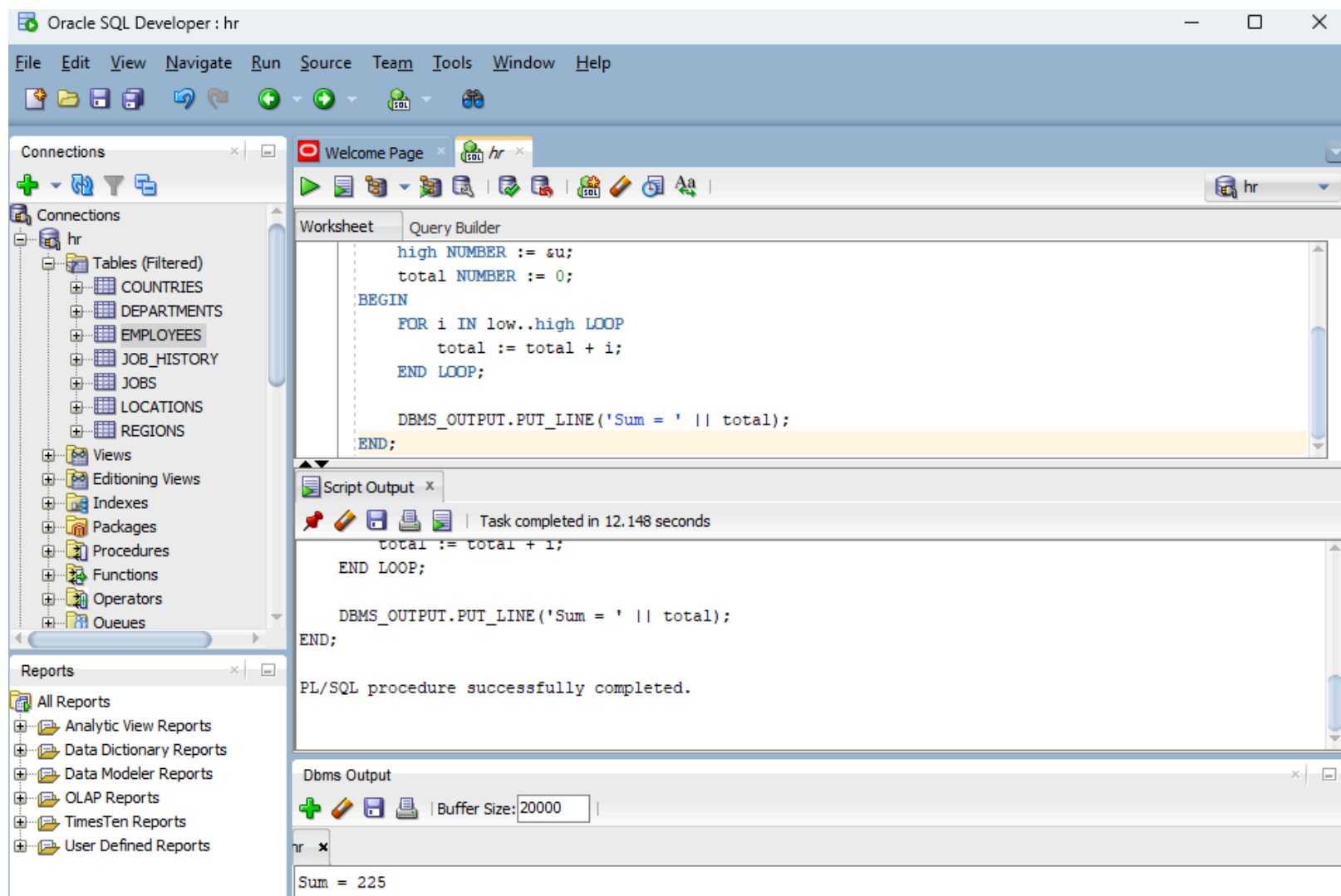
FOR i IN low..high LOOP

total := total + i;

END LOOP;

DBMS_OUTPUT.PUT_LINE('Sum = ' || total);

END;



Q11:

ACCEPT id PROMPT 'Enter employee number: '

DECLARE

v_name employees.first_name%TYPE;

v_hdate employees.hire_date%TYPE;

v_dname departments.department_name%TYPE;

BEGIN

SELECT e.first_name, e.hire_date, d.department_name

INTO v_name, v_hdate, v_dname

FROM employees e

JOIN departments d USING(department_id)

WHERE employee_id = &id;

DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);

DBMS_OUTPUT.PUT_LINE('Hire Date: ' || v_hdate);

DBMS_OUTPUT.PUT_LINE('Department: ' || v_dname);

END;

The screenshot displays the Oracle SQL Developer interface. The main window shows a PL/SQL procedure being executed. The procedure is designed to accept an employee ID and output the employee's name, hire date, and department name. The procedure is as follows:

```
ACCEPT id PROMPT 'Enter employee number: '

DECLARE
  v_name employees.first_name%TYPE;
  v_hdate employees.hire_date%TYPE;
  v_dname departments.department_name%TYPE;
BEGIN
  SELECT e.first_name, e.hire_date, d.department_name
  INTO v_name, v_hdate, v_dname
  FROM employees e
  JOIN departments d USING(department_id)
  WHERE employee_id = &id;

  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
  DBMS_OUTPUT.PUT_LINE('Hire Date: ' || v_hdate);
  DBMS_OUTPUT.PUT_LINE('Department: ' || v_dname);
END;
```

The left pane shows the database schema with tables like COUNTRIES, DEPARTMENTS, EMPLOYEES, etc. The right pane shows the execution results in the "Script Output" and "Dbms Output" windows. The "Script Output" window shows the task completed in 3.037 seconds. The "Dbms Output" window shows the output of the procedure:

```
Name: John
Hire Date: 28-SEP-05
Department: Finance
```

Q12:

ACCEPT num PROMPT 'Enter number: '

```
DECLARE
  n NUMBER := &num;
  r NUMBER := 0;
  t NUMBER := &num;
BEGIN
  WHILE t > 0 LOOP
    r := r * 10 + MOD(t, 10);
    t := FLOOR(t / 10);
  END LOOP;

  IF r = n THEN
    DBMS_OUTPUT.PUT_LINE('Palindrome');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Not Palindrome');
  END IF;
END;
```

The screenshot displays the Oracle SQL Developer interface. The main window shows a PL/SQL script in the 'Query Builder' tab. The script is designed to check if a number is a palindrome by reversing its digits. The script includes a loop to extract digits from the input number and a conditional statement to output the result.

The 'Script Output' window shows the execution progress, indicating that the task completed in 7.133 seconds. The 'Dbms Output' window shows the result of the execution: 'Palindrome'.

The 'Connections' window on the left shows the 'hr' schema selected, with a tree view of tables, views, and other database objects.

```
      t := FLOOR(t / 10);
    END LOOP;

    IF r = n THEN
      DBMS_OUTPUT.PUT_LINE('Palindrome');
    ELSE
      DBMS_OUTPUT.PUT_LINE('Not Palindrome');
    END IF;
  END;
```

Script Output x
Task completed in 7.133 seconds

```
ELSE
  DBMS_OUTPUT.PUT_LINE('Not Palindrome');
END IF;
END;
```

new: DECLARE
 n NUMBER := 6776;
 r NUMBER := 0;
 t NUMBER := 6776;
BEGIN

Dbms Output
Buffer Size: 20000

hr x
Palindrome

Q13:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  v_emp_id      employees.employee_id%TYPE;
  v_first_name   employees.first_name%TYPE;
  v_last_name    employees.last_name%TYPE;
  v_email        employees.email%TYPE;
  v_phone        employees.phone_number%TYPE;
  v_hire_date     employees.hire_date%TYPE;
  v_job_id       employees.job_id%TYPE;
  v_salary       employees.salary%TYPE;
  v_comm_pct     employees.commission_pct%TYPE;
  v_manager_id   employees.manager_id%TYPE;
  v_dept_id      employees.department_id%TYPE;
  v_dept_name    departments.department_name%TYPE;
  v_location_id  departments.location_id%TYPE;
```

```
BEGIN
```

```
  v_emp_id := &enter_employee_id;
  v_first_name := '&ENTER_FIRST_NAME';
  v_last_name := '&ENTER_LAST_NAME';
  v_email := '&ENTER_EMAIL';
  v_phone := '&ENTER_PHONE_NUMBER';
  v_hire_date := TO_DATE('&ENTER_HIRE_DATE','YYYY-MM-DD');
  v_job_id := '&ENTER_JOB_ID';
  v_salary := &enter_salary;
  v_comm_pct := &enter_commission_pct;
  v_manager_id := &enter_manager_id;
  v_dept_id := &enter_department_id;
  v_dept_name := '&ENTER_DEPARTMENT_NAME';
  v_location_id := &enter_location_id;
```

```
  INSERT INTO hr.departments (
```

```
    department_id,
    department_name,
    location_id
```

```
  ) VALUES (
```

```
    v_dept_id,
    v_dept_name,
    v_location_id
```

```
  );
```

```
  dbms_output.put_line('Department Inserted Successfully.');
```

```
  INSERT INTO hr.employees (
```

```

employee_id,
first_name,
last_name,
email,
phone_number,
hire_date,
job_id,
salary,
commission_pct,
manager_id,
department_id
) VALUES (
v_emp_id,
v_first_name,
v_last_name,
v_email,
v_phone,
v_hire_date,
v_job_id,
v_salary,
v_comm_pct,
v_manager_id,
v_dept_id
);

dbms_output.put_line('Employee Inserted Successfully.');
```

EXCEPTION

```

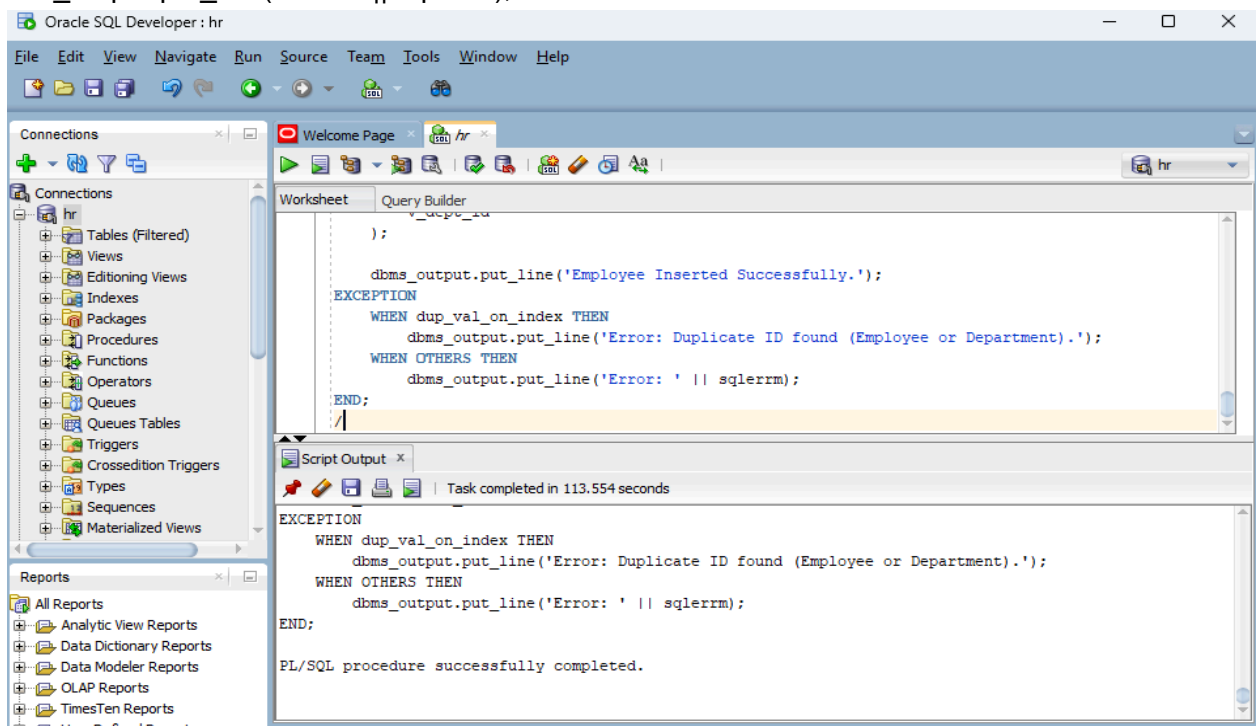
    WHEN dup_val_on_index THEN
        dbms_output.put_line('Error: Duplicate ID found (Employee or Department).');
```

WHEN OTHERS THEN

```

        dbms_output.put_line('Error: ' || sqlerrm);
END;
```

/



Q14:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_emp_id    NUMBER := 90;
```

```
    v_manager_id NUMBER;
```

```
    v_manager_sal NUMBER;
```

```
    v_manager_name VARCHAR2(100);
```

```
BEGIN
```

```
    BEGIN
```

```
        SELECT manager_id INTO v_manager_id
```

```
        FROM employees
```

```
        WHERE employee_id = v_emp_id;
```

```
    EXCEPTION
```

```
        WHEN NO_DATA_FOUND THEN
```

```
            DBMS_OUTPUT.PUT_LINE('Employee 90 does not exist.');
```

```
            RETURN;
```

```
    END;
```

```
LOOP
```

```
    IF v_manager_id IS NULL THEN
```

```
        DBMS_OUTPUT.PUT_LINE('No manager in chain earns more than 2500.');
```

```
        EXIT;
```

```
    END IF;
```

```
    SELECT salary, first_name || ' ' || last_name
```

```
    INTO v_manager_sal, v_manager_name
```

```
    FROM employees
```

```
    WHERE employee_id = v_manager_id;
```

```
    IF v_manager_sal > 2500 THEN
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            'First manager in chain earning over 2500 is: '
```

```
            || v_manager_name || ' (ID: ' || v_manager_id || ')
```

```
        );
```

```
        EXIT;
```

```
    END IF;
```

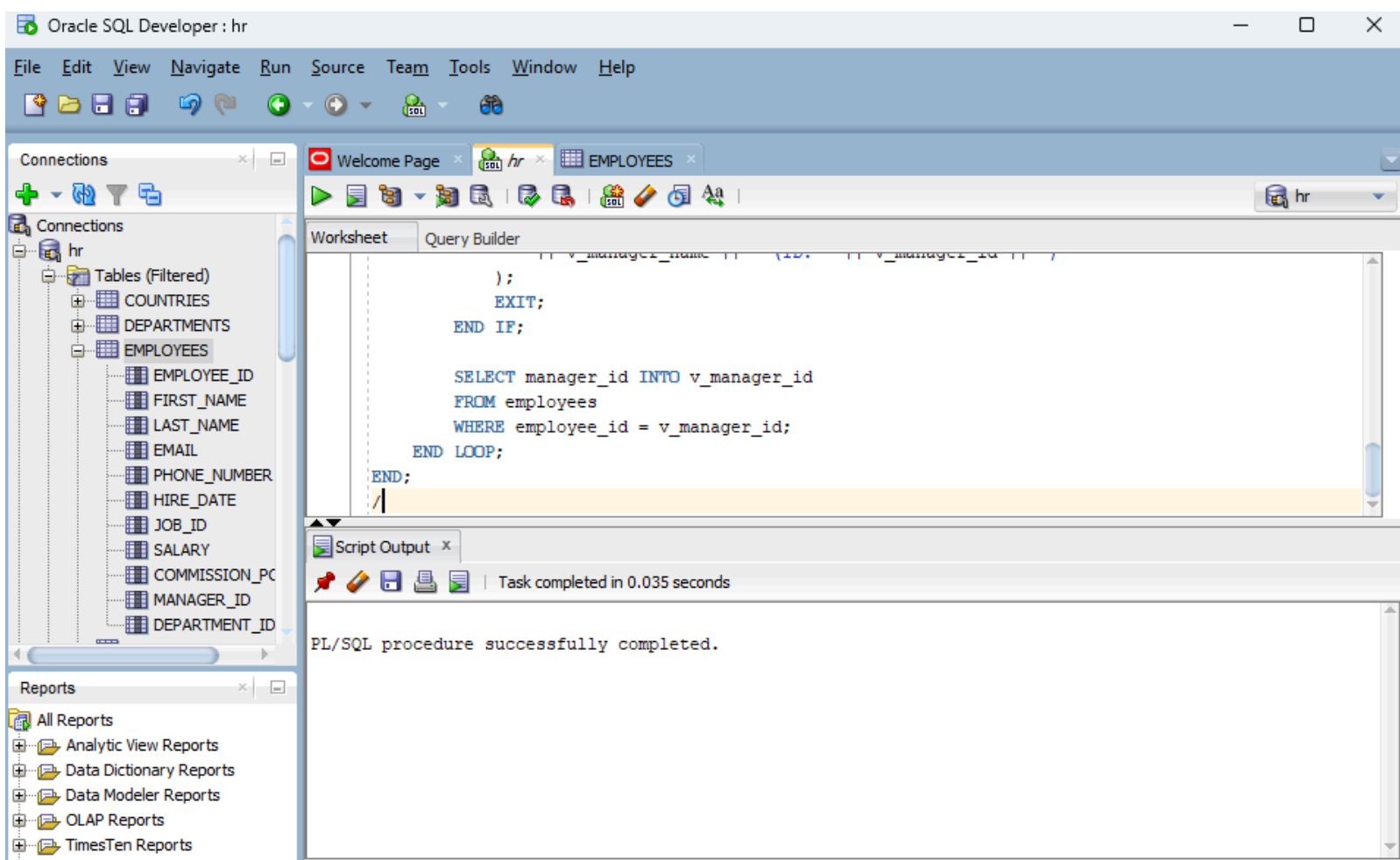
```
    SELECT manager_id INTO v_manager_id
```

```
    FROM employees
```

```
    WHERE employee_id = v_manager_id;
```

```
END LOOP;
```

```
END;  
/
```



Q15:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_sum NUMBER := 0;
```

```
    i    NUMBER := 1;
```

```
BEGIN
```

```
    WHILE i <= 100 LOOP
```

```
        v_sum := v_sum + i;
```

```
        i := i + 1;
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('Sum = ' || v_sum);
```

```
END;
```

```
/
```

The screenshot displays the Oracle SQL Developer interface. The main window shows a PL/SQL script in the 'Query Builder' tab, which is identical to the code provided in the previous blocks. The script calculates the sum of integers from 1 to 100. The 'Script Output' tab shows the message 'Task completed in 0.068 seconds'. The 'Dbms Output' tab shows the result 'Sum = 5050'. The left sidebar shows the 'Connections' pane with the 'hr' connection selected, and the 'Reports' pane with 'All Reports' selected.

Oracle SQL Developer : hr

File Edit View Navigate Run Source Team Tools Window Help

Connections

Connections

hr

Tables (Filtered)

- COUNTRIES
- DEPARTMENTS
- EMPLOYEES
 - EMPLOYEE_ID
 - FIRST_NAME
 - LAST_NAME
 - EMAIL
 - PHONE_NUMBER
 - HIRE_DATE
 - JOB_ID
 - SALARY
 - COMMISSION_PC
 - MANAGER_ID
 - DEPARTMENT_ID

Reports

All Reports

- Analytic View Reports
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- TimesTen Reports
- User Defined Reports

Welcome Page hr EMPLOYEES

Worksheet Query Builder

```
v_sum NUMBER := 0;
i    NUMBER := 1;

BEGIN
    WHILE i <= 100 LOOP
        v_sum := v_sum + i;
        i := i + 1;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum = ' || v_sum);
END;
/
```

Script Output

Task completed in 0.068 seconds

PL/SQL procedure successfully completed.

Dbms Output

Buffer Size: 20000

hr

Sum = 5050